PDV WEB · DOCUMENTAÇÃO DE TESTES

Plano de Testes

Campo	Valor
Identificador	PT-PDV-001
Versão	1.0
Data	26/10/2026
Projeto	Sistema de Ponto de Venda (PDV) Web
Equipe do projeto	Hugo de Oliveira Guimarães, Jean Wictor, Cesar Franco
Modelo seguido	IEEE 829 / ISO/IEC/IEEE 29119-3 (Plano de Teste), processo de 7 etapas do *Tutorial de Cenários de Testes*
Status	Aprovado para execução — Ciclo 1 executado (ver Relatório de Execução)

1. Introdução

Este plano define **o que** será testado no PDV Web, **como**, **com quais critérios** e **com quais entregáveis**. O objetivo é mostrar, com evidência objetiva e reproduzível, que o backend `pdv-web-api` atende às regras de negócio, às validações VAL-01 a VAL-08 e ao contrato da API v2.3. O plano também cobre o reteste das correções feitas no sistema desktop legado.

Objetivos específicos:

1. Verificar os 16 casos de uso (UC01–UC16) e as regras de sistema (Regras 1.1, 1.2, 2.1–2.3).
2. Garantir a **integridade financeira e de estoque**, os riscos centrais do Plano de Projeto (seção 1.9): vendas sem saldo, vendas concorrentes, descontos sem autorização e estornos.
3. Garantir o **controle de acesso por perfil** (VAL-02) e a segurança da autenticação (VAL-06).
4. Medir o tempo de resposta contra o limite de 3 segundos (VAL-07).
5. Produzir evidências e métricas que sustentem a decisão de seguir para a fase Beta.

2. Documentos de referência

Ref.	Documento	Versão/Data
R1	Documentação Técnica da API RESTful	v2.3 — 13/09/2026
R2	Documentação de Casos de Uso — PDV Web	09/2026
R3	Dicionário de Dados	13/09/2026
R4	Documentação do Projeto PDV Web	24/08/2026
R5	Regras de Sistema / Tabela de Planejamento e Regras	17/08/2026
R6	Plano de Projeto	17/08/2026
R7	Tutorial — Criação de Cenários de Testes no Desenvolvimento de Software	—
R8	ADR-001 — Decisões de arquitetura e resolução de inconsistências	26/10/2026

Onde os documentos R1–R6 divergem, vale a regra de precedência do ADR-001 (R8).

3. Itens de teste

Item	Versão	Descrição
`pdv-web-api`	2.3.0	Backend REST (Java 25, Spring Boot 4.1, PostgreSQL 17)
Esquema do banco	Flyway V1, V2	Tabelas, restrições (CHECK, UNIQUE, FK) e dados iniciais
`SistemaVendasEmpresa` (desktop)	1.0-SNAPSHOT	Sistema legado JavaFX — apenas reteste das correções DEF-DSK

4. Escopo

4.1 Funcionalidades a testar

Grupo	Requisitos	Prioridade
Autenticação e sessão	UC01, VAL-06, Regra 2.3 (login obrigatório), política JWT 24 h	P1
Controle de acesso (RBAC)	VAL-02, Regra 2.2 (restrição de perfil)	P1
Registro de venda e estoque	UC04, UC05, VAL-01, VAL-08 (reserva de estoque — parcial, ADR D-02), Regra 1.1, Regra 2.1	P1
Descontos	UC06, UC07, Regra 1.1 (cupom / autorização)	P1
Cancelamento e estorno	UC11, VAL-03, Regra 2.2	P1
Controle de caixa	UC02, UC03, UC15, VAL-05	P1
Alerta de estoque crítico	VAL-04, UC09	P2
Comissões e histórico	UC10, Regra 1.1.1.11	P2
Cadastros	UC08 (clientes), UC12 (produtos), UC13 (usuários), UC14 (cupons)	P2
Auditoria	UC16, Regra 1.2	P2
Contrato da API	Paginação, RFC 7807, snake_case, PUT×PATCH, soft delete, imutabilidade de vendas (R1, seções 1–2)	P2
Desempenho	VAL-07 (< 3 s)	P2
Integridade do banco	Restrições do esquema e migrações	P2
Correções do desktop	DEF-DSK-001 a DEF-DSK-008	P3

4.2 Fora do escopo deste ciclo

Item	Motivo	Tratamento
Interface Web (front-end)	Ainda não desenvolvida	Testes E2E de interface (Playwright) planejados para o próximo ciclo
Alerta "em tempo real nas telas" (push)	Depende do front-end (ADR D-11)	Backend testado (flag e lista de alertas)
Implantação Docker / AWS EC2	Docker indisponível no ambiente de teste	Pendência PEND-03
Teste de carga em produção (JMeter)	Requer o ambiente AWS	Pendência PEND-04; aqui só a medição local VAL-07
Emissão de NF-e/NFC-e e integração TEF	Fora do escopo do projeto (R6, seção 1.6)	—
Usabilidade	Sem interface	Próximo ciclo

5. Abordagem

5.1 Processo (Tutorial, seção 3)

Etapa	Como foi aplicada	Artefato
1. Entender os requisitos	Leitura de R1–R6, extração das regras e resolução de conflitos	ADR-001
2. Identificar condições	Regras → condições de teste por requisito	Especificação, seção 1
3. Definir cenários	Positivos, negativos, limites, exceção, segurança, desempenho e regressão	Especificação, seção 2
4. Preparar os casos	Casos com ID, requisito, dados, passos e resultado esperado	Especificação, seções 3–4
5. Priorizar	P1–P4 por risco × impacto (seção 7 deste plano)	Coluna "Prior." do catálogo
6. Executar e registrar	Execução automatizada com evidências em XML/CSV/HTML	Relatório de Execução
7. Corrigir e retestar	Defeitos registrados, corrigidos e regressão completa	Relatório de Defeitos

5.2 Níveis de teste (pirâmide)

Nível	Objetivo	Ferramentas	Isolamento
Unitário	Regras de negócio puras (cálculo, validação, estados) e serviços com dependências simuladas	JUnit 6 (Jupiter), Mockito, AssertJ	Sem banco e sem rede
Componente (API)	Contrato HTTP: status, formato RFC 7807, validação de entrada, autorização	`@WebMvcTest`, MockMvc, `@MockitoBean`, spring-security-test	Serviço simulado
Integração	Fluxo HTTP → segurança → serviço → JPA → PostgreSQL real, incluindo transações e restrições	`@SpringBootTest`, MockMvc, PostgreSQL 17 embarcado (Zonky)	Banco exclusivo, limpo a cada teste
Sistema (API ponta a ponta)	Fluxos completos de negócio: abrir caixa → vender → sangria → cancelar → fechar	Mesmos da integração	—
Aceitação (BDD)	Regras escritas em linguagem de negócio (Gherkin pt-BR) e executáveis	Cucumber 7 + JUnit Platform Suite	—

5.3 Tipos de teste e técnicas

Tipo / técnica (Tutorial)	Onde é aplicado
Positivo / negativo (4.1, 4.2)	Todos os módulos
Valores-limite (4.3, 5.2)	Preço, estoque, quantidade, comissão, percentual de cupom, vigência, senha, paginação, sangria, desconto vs. restante, arredondamento
Particionamento de equivalência (5.1)	CPF/CNPJ, tipos de pessoa, modalidades de pagamento, perfis
Tabela de decisão (5.3)	Cupom × desconto direto × perfil × autorização
Baseado em estados (5.4)	Caixa (ABERTO → FECHADO) e Venda (CONCLUIDA → CANCELADA)
Exceção (4.4)	Banco indisponível, erro inesperado, JSON malformado, recurso inexistente
Segurança (4.5)	Sem token, token adulterado, outra chave, perfil sem permissão, IDOR, injeção, usuário inativado
Desempenho (4.6)	VAL-07 — p95 de consultas e vendas
Concorrência	Venda simultânea do último item, cancelamento duplo, sangrias e aberturas simultâneas
Regressão (4.7)	Bateria completa a cada correção (`mvnw verify`)
Exploratório (5.6)	Roteiro com sessões planejadas — 07-roteiro-de-testes-manuais.md

Tipo / técnica (Tutorial)	Onde é aplicado
Teste de mutação (complemento ao 14)	PIT sobre as regras de negócio, para medir a eficácia das asserções

5.4 Dados de teste

- **CPF/CNPJ sintéticos**, gerados com dígitos verificadores válidos (`GeradorDocumentos`). Nenhum documento de pessoa real é usado (LGPD).
- O banco é truncado antes de cada teste e recebe só o Consumidor Padrão e um gerente de teste: os resultados não dependem da ordem de execução.
- Massa de desempenho: 1.000 produtos gerados por `generate_series` no PostgreSQL.
- Exemplos oficiais usados como dados: tabela da seção 3 das Regras de Sistema (Camiseta, Tênis 41, Meia) e exemplos de payload da API v2.3 (venda de R$ 215,00, comissão de R$ 225,00, saldo de caixa de R$ 300,00).

6. Critérios

6.1 Critérios de entrada

- Código compila (`mvnw test-compile`) e as migrações Flyway aplicam sem erro.
- Requisitos com divergência resolvida (ADR-001).
- Ambiente de teste disponível (JDK 25 e binários do PostgreSQL embarcado).

6.2 Critérios de saída (aceite do ciclo)

#	Critério	Meta
CS-1	Casos P1 executados e aprovados	100%
CS-2	Taxa de aprovação geral	≥ 95%
CS-3	Defeitos de severidade Crítica ou Alta em aberto	0
CS-4	Requisitos com ao menos um caso executado e aprovado	100% dos requisitos no escopo (4.1)
CS-5	Cobertura de código (JaCoCo)	≥ 80% de linhas e ≥ 70% de ramos (verificado automaticamente no `verify`)
CS-6	Tempo de resposta (VAL-07)	p95 < 3.000 ms em todas as medições
CS-7	Força dos testes unitários (mutantes mortos ÷ mutantes cobertos)	≥ 80%

6.3 Critérios de suspensão e retomada

Suspensão	Retomada
Build não compila ou migração falha	Correção integrada e build verde
Mais de 20% dos casos falhando por um mesmo defeito bloqueante	Defeito corrigido e reteste do bloco
Ambiente de banco indisponível	Ambiente restabelecido

7. Priorização (Tutorial, seção 13)

Prioridade	Interpretação	Exemplos no PDV
P1 — Crítica	Integridade financeira, de estoque ou de acesso	Bloqueio por saldo, concorrência, descontos, cancelamento, RBAC, login, caixa

Prioridade	Interpretação	Exemplos no PDV
P2 — Alta	Impede funções principais	Cadastros, comissões, auditoria, contrato da API, desempenho
P3 — Média	Há alternativa de contorno	Filtros de busca, mensagens, correções do desktop
P4 — Baixa	Baixo impacto	Normalizações cosméticas (maiúsculas, máscara)

8. Ambiente de teste

Componente	Versão
Sistema operacional	Windows 10 Pro 10.0.19045
JDK	OpenJDK 25.0.3 (JetBrains Runtime)
Build	Maven 3.9.16 (Maven Wrapper 3.3.4)
Framework	Spring Boot 4.1.2
Banco	PostgreSQL 17 embarcado (Zonky embedded-postgres 2.2.2, binários 17.11.0)
Testes	JUnit Jupiter 6.0.3, Mockito 5.23, AssertJ, MockMvc, spring-security-test, Cucumber 7.34.9, JaCoCo 0.8.15, PIT 1.30.1

9. Riscos do processo de teste

Risco	Prob.	Impacto	Mitigação
Sem Docker na máquina de testes	Ocorreu	Médio	PostgreSQL embarcado real nos testes de integração; Docker fica como PEND-03
Diferença entre o ambiente local (Windows) e a produção (Linux/AWS)	Média	Médio	`.gitattributes` para LF nos scripts; pipeline Linux criado em `.github/workflows/ci.yml` (PEND-05, roda no primeiro push)
Requisitos conflitantes entre documentos	Ocorreu	Alto	ADR-001 com precedência explícita; D-04 (base da comissão) depende de confirmação
Front-end inexistente	Certa	Médio	Testes concentrados na API; E2E de interface no próximo ciclo
Falsa sensação de qualidade pela cobertura	Média	Alto	Teste de mutação (PIT) para medir a força das asserções
Testes instáveis (concorrência / tempo)	Baixa	Médio	Largada sincronizada (`CountDownLatch`), medição com aquecimento e p95

10. Papéis e responsabilidades

Papel	Responsabilidade
Equipe de desenvolvimento	Implementar, automatizar os testes e corrigir defeitos
Revisor de testes (rodízio na equipe)	Revisar os casos e as evidências; aprovar o relatório de execução
Gerentes de loja (partes interessadas)	Validar os cenários BDD (linguagem de negócio) na fase Beta

11. Cronograma de testes

Ciclo	Conteúdo	Data
Ciclo 1	Backend: unitário, API, integração, BDD, segurança, concorrência, desempenho local, mutação	26/10/2026 — **executado**

Ciclo	Conteúdo	Data
Ciclo 2	Front-end: E2E de interface, usabilidade, push de alertas	Após a entrega do front-end
Ciclo 3	Implantação Docker/AWS, carga com JMeter, testes exploratórios em ambiente de homologação	Antes da Beta (01/03/2027)
Aceitação	Operação assistida em loja (Beta)	01/03/2027 – 30/06/2027

12. Gestão de defeitos

Ciclo de vida: **Novo** → **Em análise** → **Corrigido** → **Retestado** → **Fechado** (ou **Reaberto**).

Severidade	Definição
Crítica	Perda ou corrupção de dados, prejuízo financeiro ou falha de segurança explorável
Alta	Função principal indisponível, sem contorno
Média	Comportamento incorreto com contorno, ou falha de segurança de baixo impacto
Baixa	Cosmético, documentação ou manutenção

Todo defeito corrigido exige: (1) reteste do caso que o encontrou e (2) regressão completa (`mvnw clean verify`).

13. Entregáveis

Entregável	Local
Plano de Testes (este documento)	`docs/testes/01-plano-de-testes.md`
Especificação de casos de teste	`docs/testes/02-especificacao-de-casos-de-teste.md`
Cenários BDD (Gherkin)	`docs/testes/03-cenarios-bdd.md` e `pdv-web-api/src/test/resources/features/`
Matriz de rastreabilidade	`docs/testes/04-matriz-de-rastreabilidade.md`
Relatório de execução	`docs/testes/05-relatorio-de-execucao.md`
Relatório de defeitos	`docs/testes/06-relatorio-de-defeitos.md`
Roteiro de testes manuais e exploratórios	`docs/testes/07-roteiro-de-testes-manuais.md`
Evidências da execução	`docs/testes/evidencias/`
Testes automatizados	`pdv-web-api/src/test/`

14. Aprovação

Papel	Nome	Data	Assinatura
Responsável pelo projeto			
Revisor de testes			